Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😁

alkemy

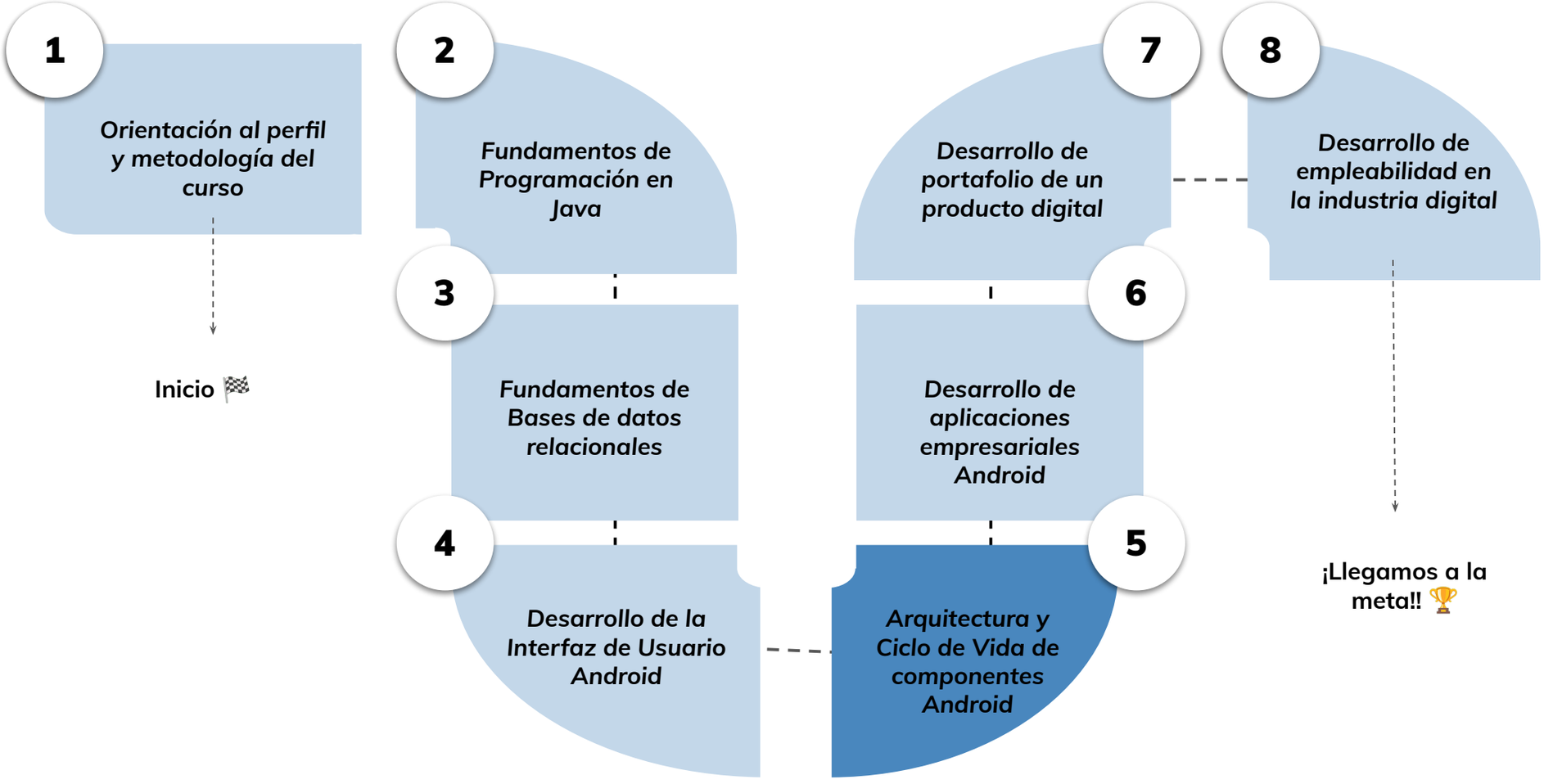
Clase grabada

Patrones de arquitectura en Android - Parte I

AE: Utilizar patrones de arquitectura escalables para la construcción de una aplicación android de acuerdo a los requerimientos.

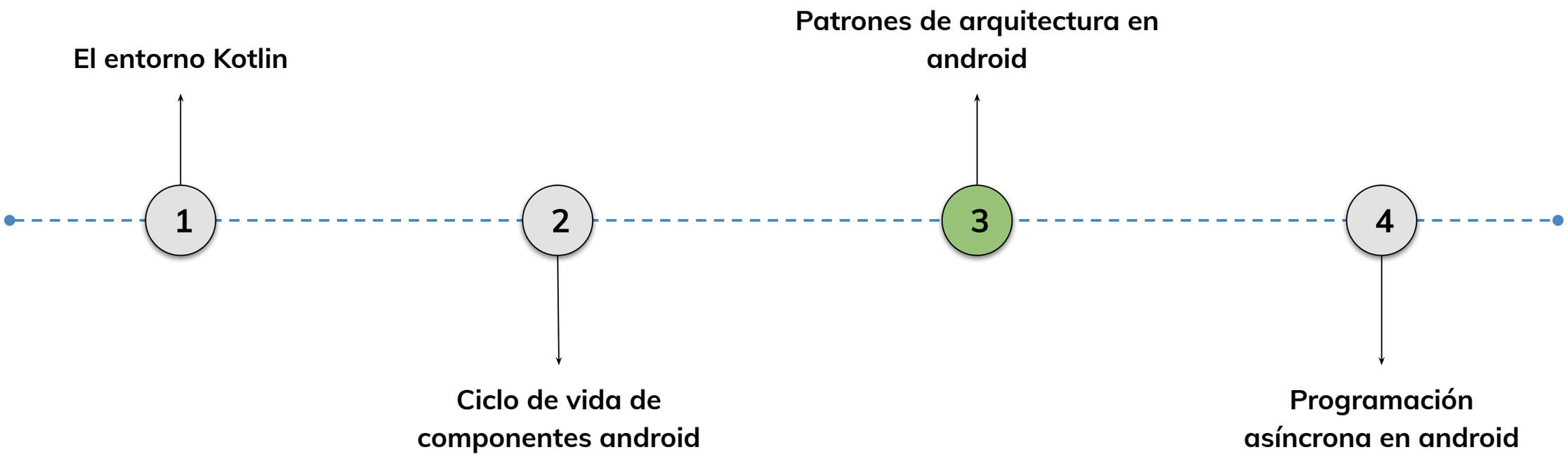
Hoja de ruta

*¿Cuáles **módulos** conforman el programa?*



Roadmap de la lección

*¿Cuáles **lecciones** estaremos estudiando en este módulo?*



Learning Path

¿Cuáles temas trabajaremos hoy?

5.

Patrones de arquitectura

- Patrones de Arquitectura
- Patrones de diseño
- Tipos de Patrones de diseño
- Patrones de arquitectura

- ¿Qué son?
- Ventajas
- Patrones comunes

alkemy

Objetivos de aprendizaje

¿Qué aprenderás?

- ***Comprender qué son los patrones de arquitectura***
- ***Entender los Patrones de arquitectura***
- ***Conocer el Patron MVC***
- ***Entender qué son los patrones de diseño***
- ***Comprender qué tipos de diseño existen***
- ***Conocer los patrones de arquitectura más comunes***

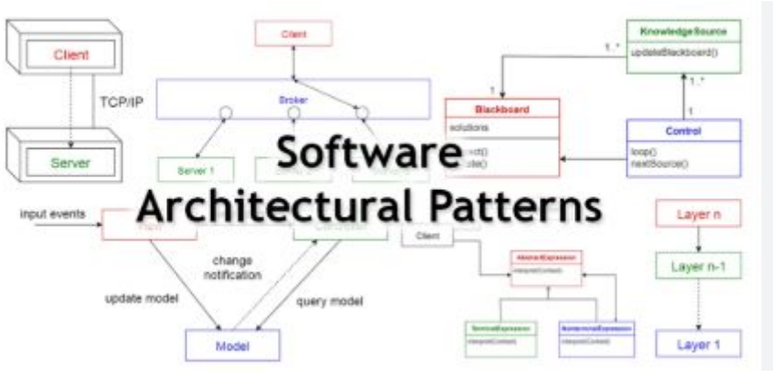
Rompehielo🥶

Contexto: 🙌

Existen múltiples tipos de patrones de diseño que se usan en todos los lenguajes de programación y son comunes en lenguajes como Java y Python

Consigna: ✍️

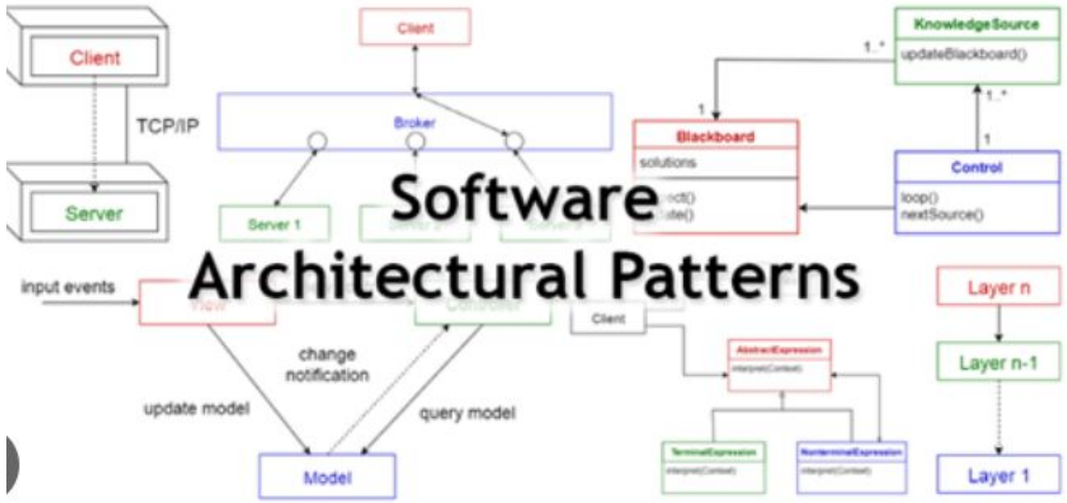
¿Alguna vez usaron alguno?¿Cuáles?



› Patrones de arquitectura

Patrones de arquitectura

1. Es una forma de organizar el código.
2. Es un concepto que captura elementos esenciales de una arquitectura.
3. Ofrece soluciones a problemas de [arquitectura de software](#).
4. Entrega una **descripción de los elementos y tipos de relaciones** que tienen.
5. Entrega un **conjunto de restricciones** sobre cómo puede ser usados.



Ventajas

Los **patrones de arquitectura en Android** ofrecen varias ventajas clave:

1. **Mantenimiento fácil**: Código organizado y fácil de modificar.
2. **Escalabilidad**: Facilita el crecimiento controlado de la aplicación.
3. **Pruebas eficientes**: Mejora la testabilidad gracias al desacoplamiento de componentes.
4. **Reutilización de código**: Reduce la duplicación y mejora la consistencia.
5. **Desacoplamiento**: Las diferentes partes de la app son independientes y fáciles de cambiar.
6. **Trabajo en equipo**: Facilita la colaboración entre desarrolladores.
7. **Integración de nuevas tecnologías**: Permite incorporar nuevas herramientas sin afectar el sistema.

Ventajas

Mantener la Organización y Estructura del Código:

Los patrones de arquitectura ayudan a mantener el código de una aplicación Android organizado y estructurado de una manera que sea fácil de entender y mantener. Esto es especialmente importante en aplicaciones grandes y complejas.

Ventajas

Separación de Responsabilidades:

Los patrones de arquitectura fomentan la separación de responsabilidades, lo que significa que diferentes partes del código se encargan de tareas específicas. Esto facilita la identificación y corrección de errores, así como la expansión de la funcionalidad.Esto también es muy útil en el trabajo colaborativo en general

Ventajas

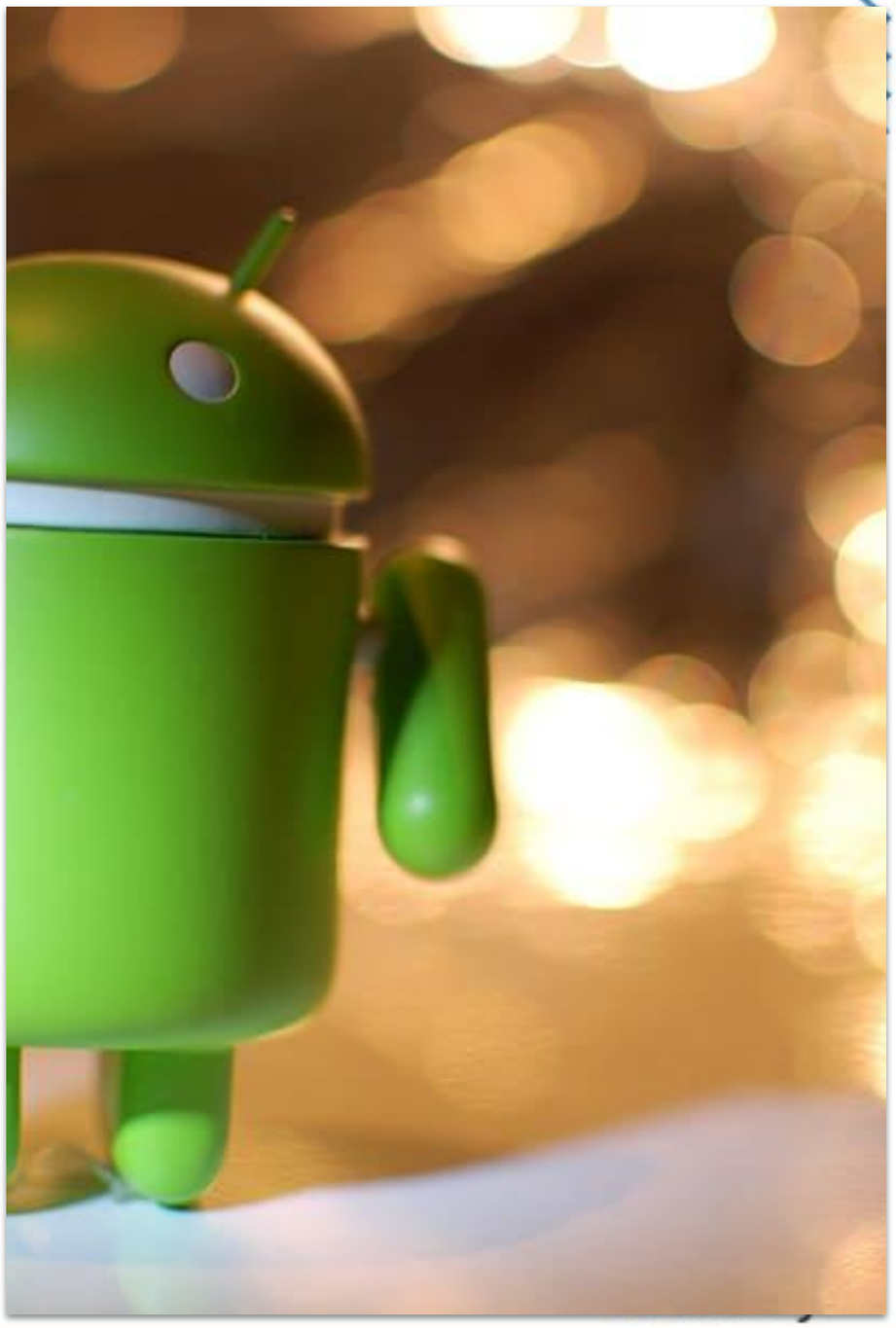
Facilitar la Prueba (Testing):

Muchos patrones de arquitectura, como el Modelo-Vista-Controlador (MVC), el Modelo-Vista-Presentador (MVP) y el Modelo-Vista-ViewModel (MVVM), facilitan la escritura de pruebas unitarias y de integración, lo que mejora la calidad del software en general.

Ventajas

Escalabilidad:

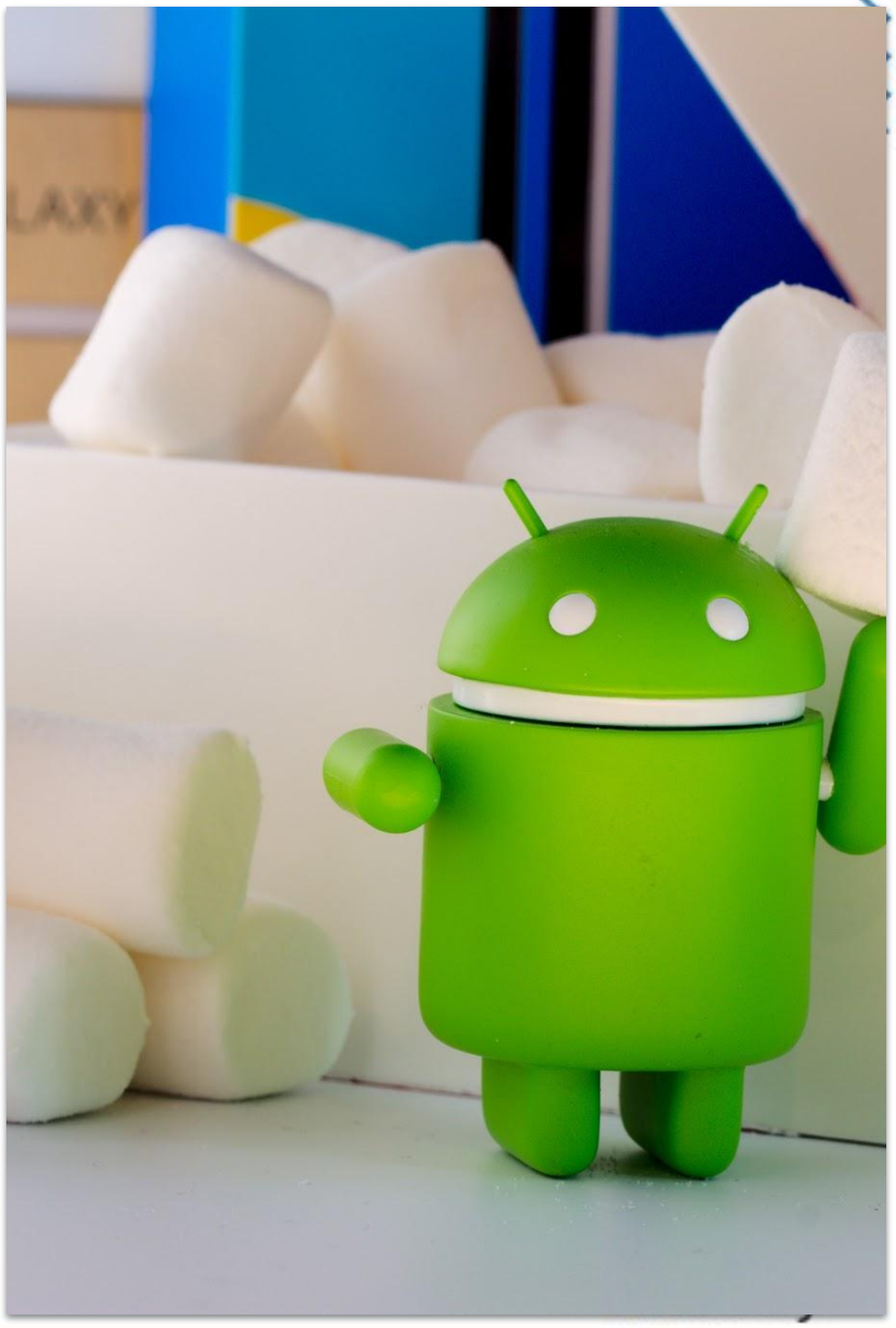
Los patrones de arquitectura están diseñados para permitir que una aplicación se expanda y evolucione de manera sostenible a medida que se agregan nuevas características o se hacen cambios en el código existente.



Ventajas

Reutilización de Código:

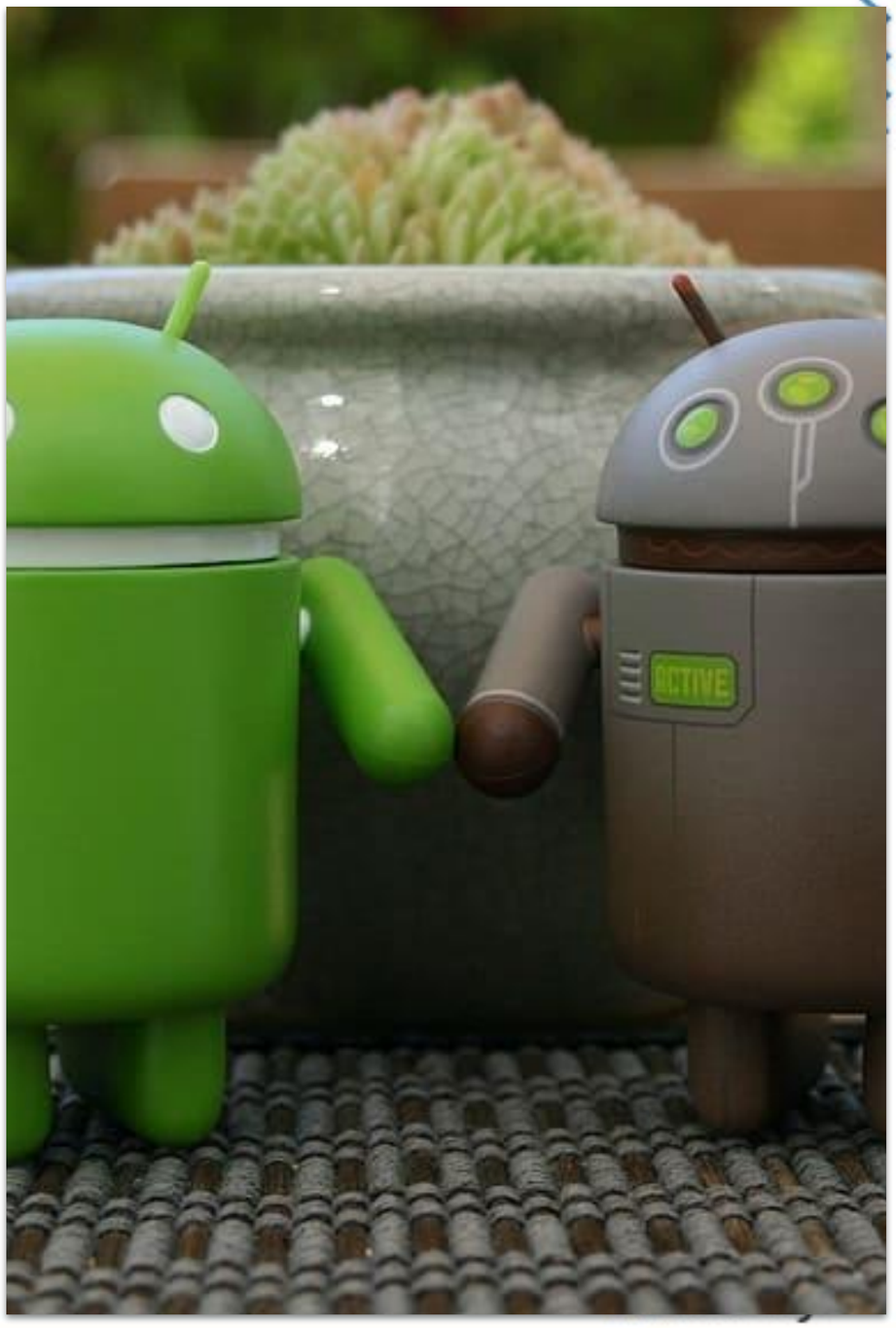
Estos patrones fomentan la reutilización de código, lo que significa que puedes usar componentes de tu aplicación en diferentes partes de la misma aplicación o incluso en otras aplicaciones si es necesario



Ventajas

Mantenimiento Simplificado:

La estructura y organización que proporcionan los patrones de arquitectura hacen que el mantenimiento y la resolución de problemas sean más simples y menos propensos a errores.

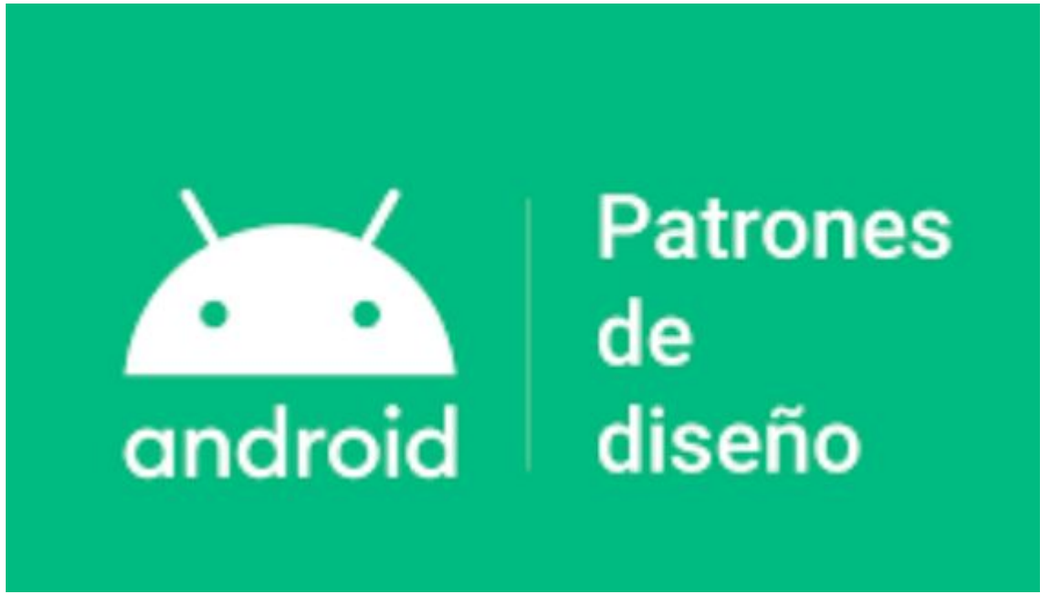


› Patrones de diseño

Patrones de diseño

¿Qué son?:

En Android, se utilizan varios patrones de diseño, además de las arquitecturas mencionadas anteriormente, para abordar diversos aspectos del desarrollo de aplicaciones móviles. Aquí tienes una lista de algunos de los patrones de diseño comunes utilizados en Android:

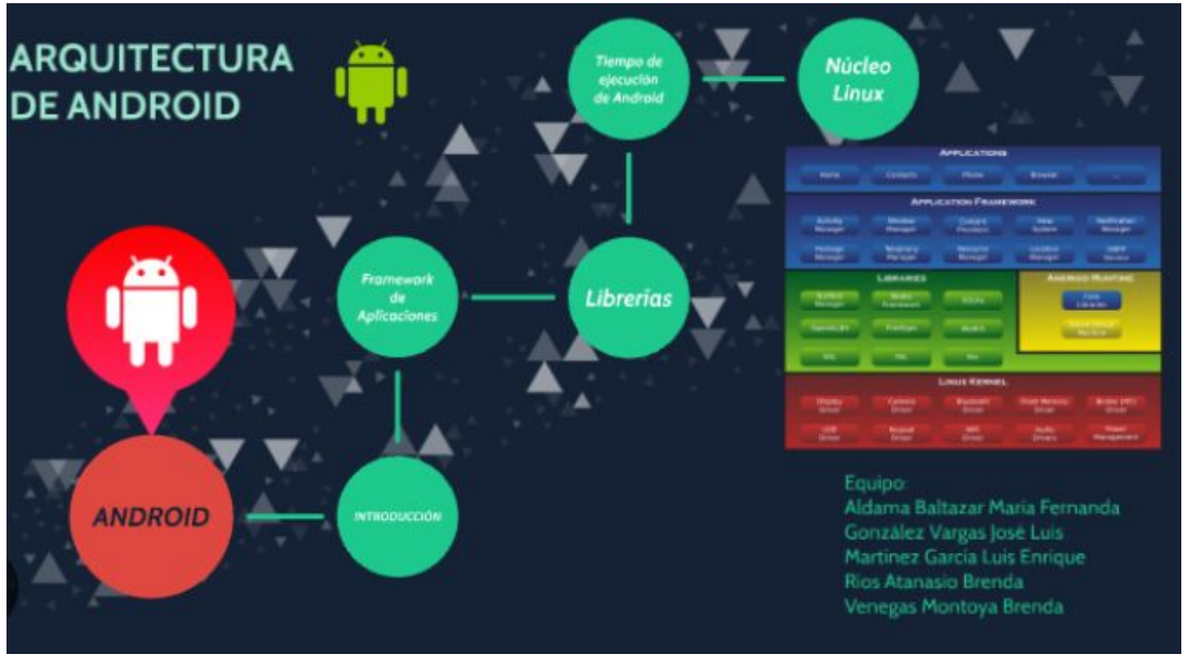


› Tipos de patrones de diseño

Tipos de patrones de diseño

¿Qué son?:

Los patrones de diseño se pueden clasificar por múltiples características, comúnmente elegimos diferenciarlos por su propósito y responsabilidad en el diseño de software en general,
En la lección anterior hablamos de algunos de los más comunes patrones pero en esta lección vamos a explorar los más representativos de cada tipo de patrón



Tipos de patrones de diseño

Un **patrón de diseño de arquitectura** es una solución reutilizable y probada para organizar y estructurar una aplicación de software de manera eficiente. Define la manera en que se distribuyen las responsabilidades y se comunican los diferentes componentes dentro de una aplicación, con el objetivo de hacerla más **mantenible**, **escalable**, **testable** y **modular**.

En el contexto de Android, los patrones de diseño de arquitectura son utilizados para mejorar la organización del código y facilitar su evolución. Algunos ejemplos comunes son:

- **MVC (Model-View-Controller)**: Divide la aplicación en tres componentes: el **Modelo** (datos), la **Vista** (interfaz de usuario) y el **Controlador** (gestiona la lógica de la aplicación).
- **MVP (Model-View-Presenter)**: Similar al MVC, pero el **Presentador** maneja la lógica de la vista, lo que permite un mejor desacoplamiento.
- **MVVM (Model-View-ViewModel)**: La **Vista** se comunica con el **ViewModel**, que contiene la lógica de presentación, mientras que el **Modelo** maneja los datos.
- **Clean Architecture**: Promueve una estructura más modular y flexible, separando las capas de la aplicación (interfaz de usuario, lógica de negocio, acceso a datos, etc.) para que sean independientes entre sí.

¿Alguna consulta?

alkemy

Momento:

Time-out!

⏰5 -10 min.

alkemy

› Patrones de arquitectura

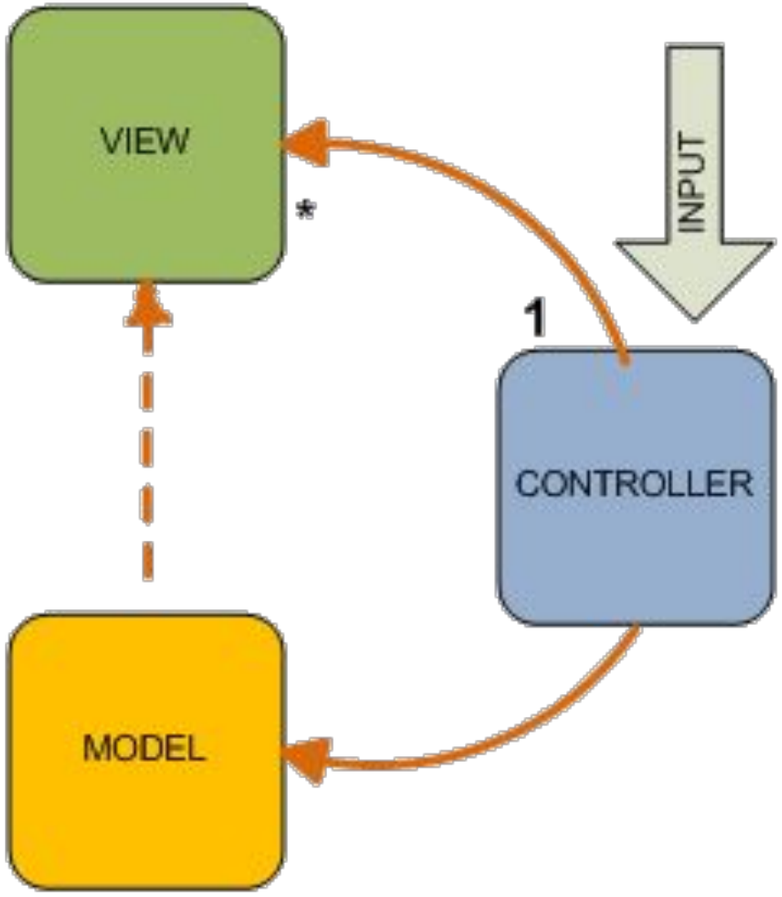
Patrones de arquitectura recomendados

¿Cuáles son?:

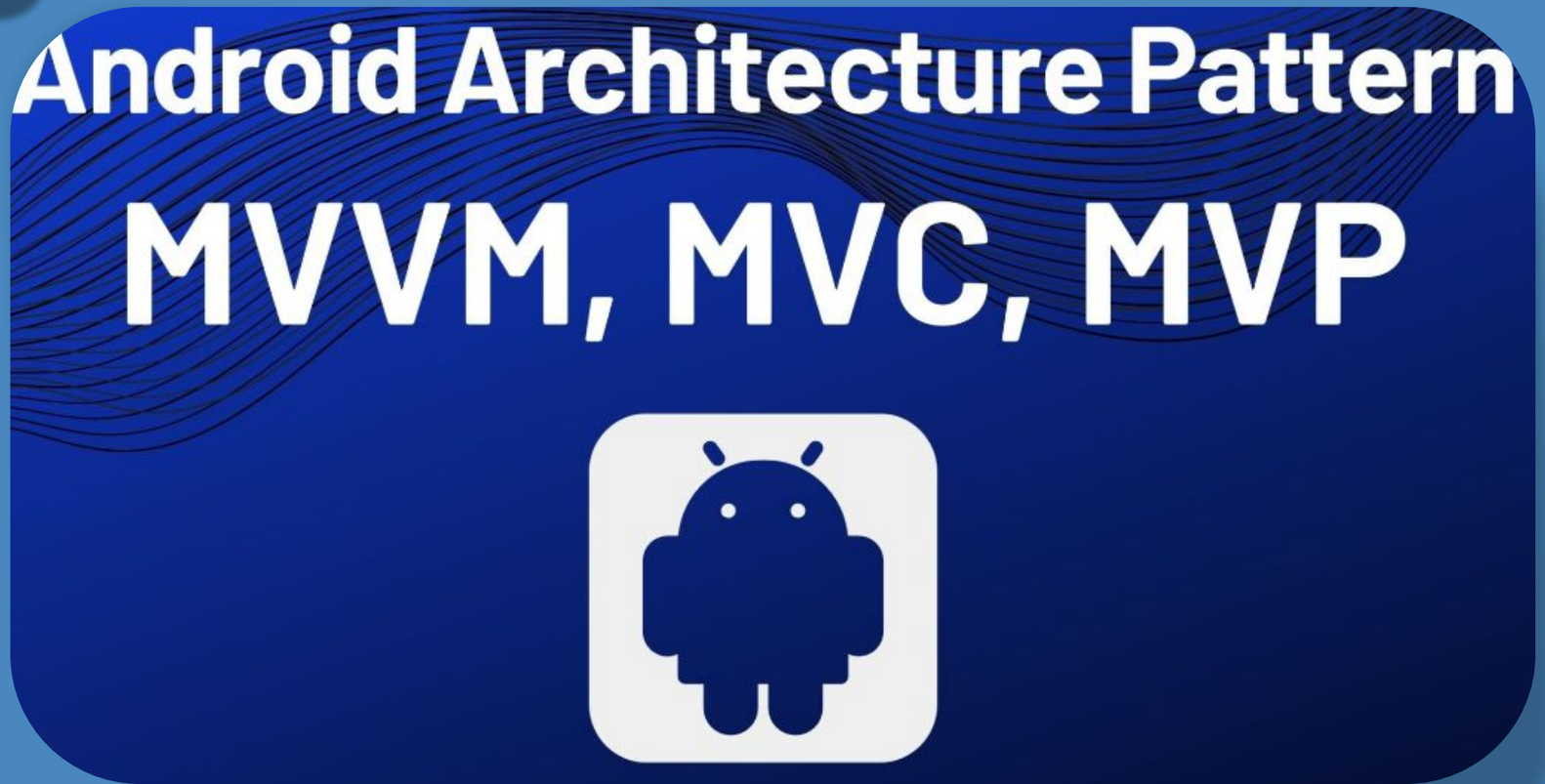
Para el desarrollo de aplicaciones Android, existen varios patrones de arquitectura recomendados que ayudan a organizar el código, mejorar la mantenibilidad y escalabilidad, y promover las mejores prácticas de desarrollo.
Actualmente dependiendo del proyecto o prácticas de una compañía podemos ver diferentes patrones en uso pero en general lo que parecen tener mejor compatibilidad con los componentes y requerimientos de Android son los siguiente:

Modelo-Vista-Controlador (MVC)

Este es uno de los patrones más antiguos y simples. En Android, la Vista se refiere a la interfaz de usuario (generalmente una actividad o fragmento), el Modelo representa los datos y la lógica de la aplicacin, y el Controlador actúa como intermediario entre la Vista y el Modelo. Sin embargo, MVC a menudo conduce a óactividades o fragmentos sobrecargados con lógica por lo que cayó en desuso en los últimos años



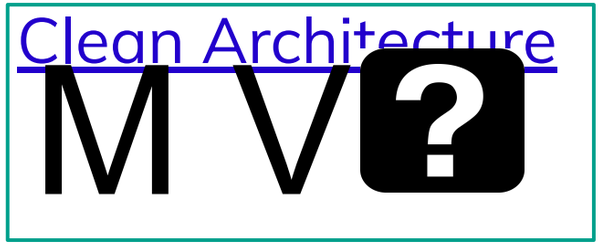
PATRONES DE ARQUITECTURA EN ANDROID

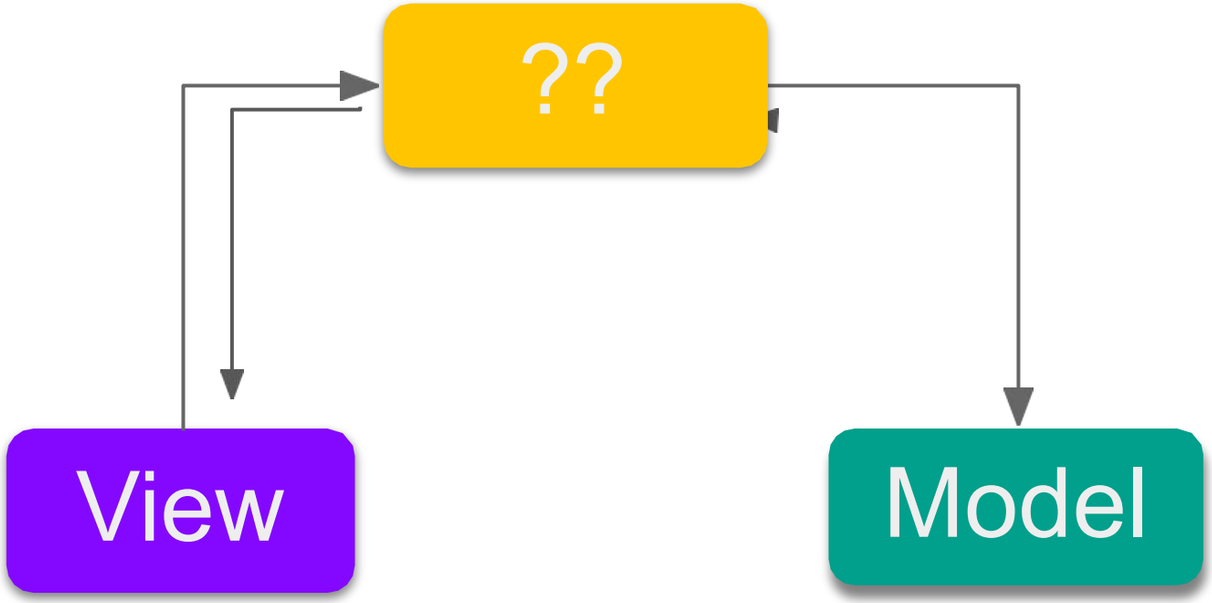


Patrones de Arquitectura en Android

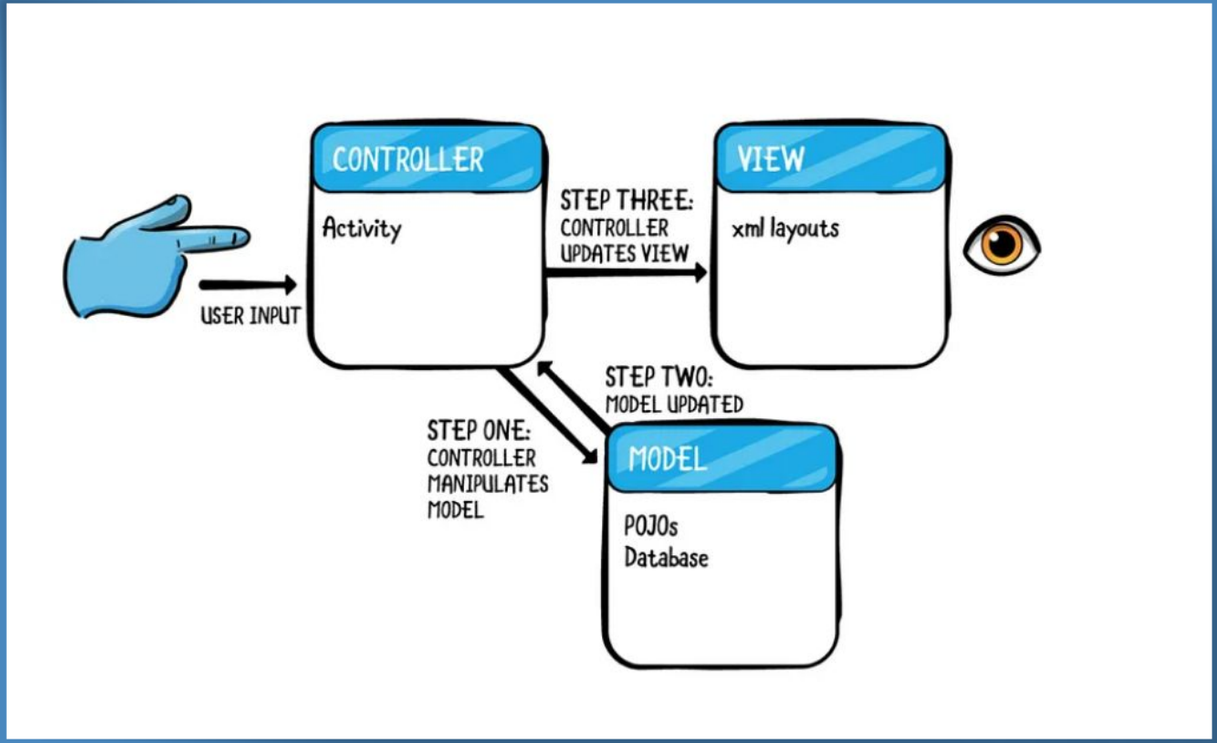
Patrones de arquitectura comunes en Android

- Model-View-Controller
- Model-View-Presenter
- Model-View-ViewModel
- Model-View-Intent
- Clean Architecture

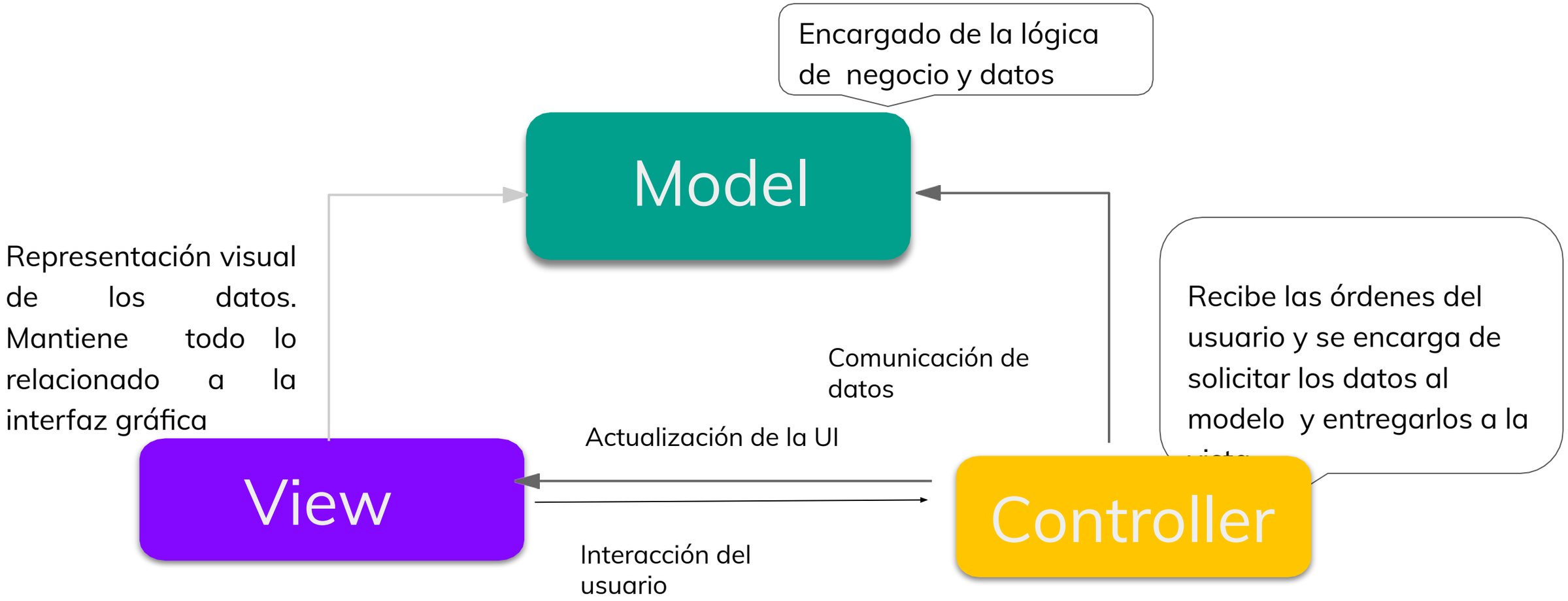




MVC
MODELO VISTA CONTROLADOR

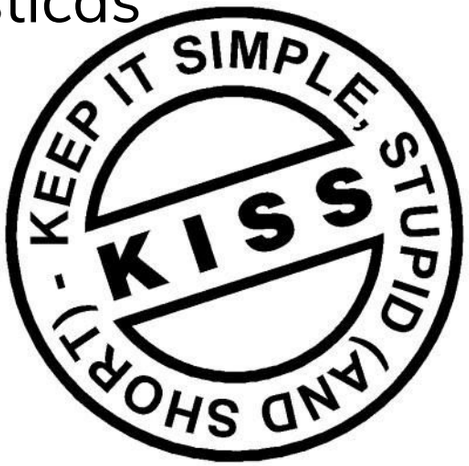


MODELO VISTA CONTROLADOR

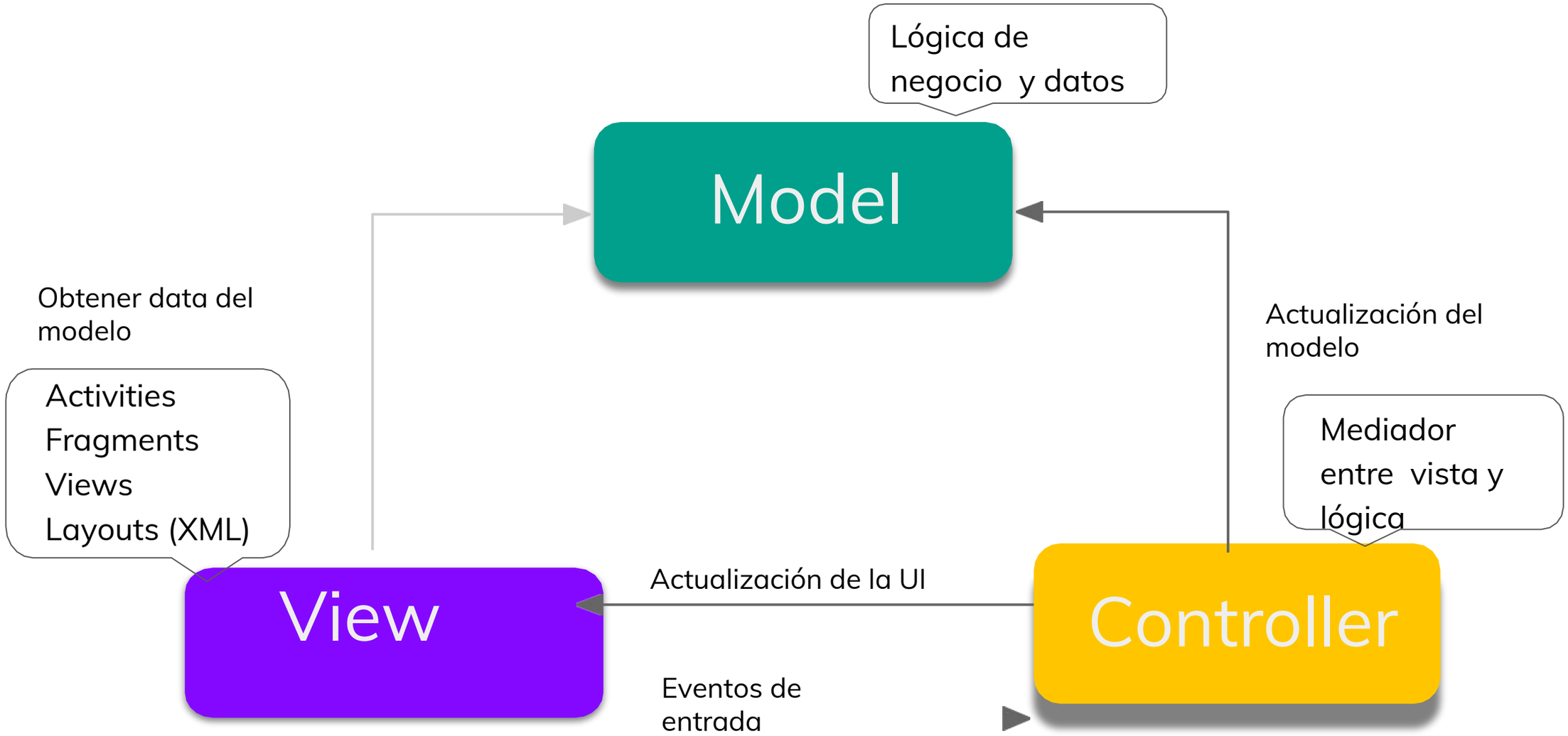


MVC en Android

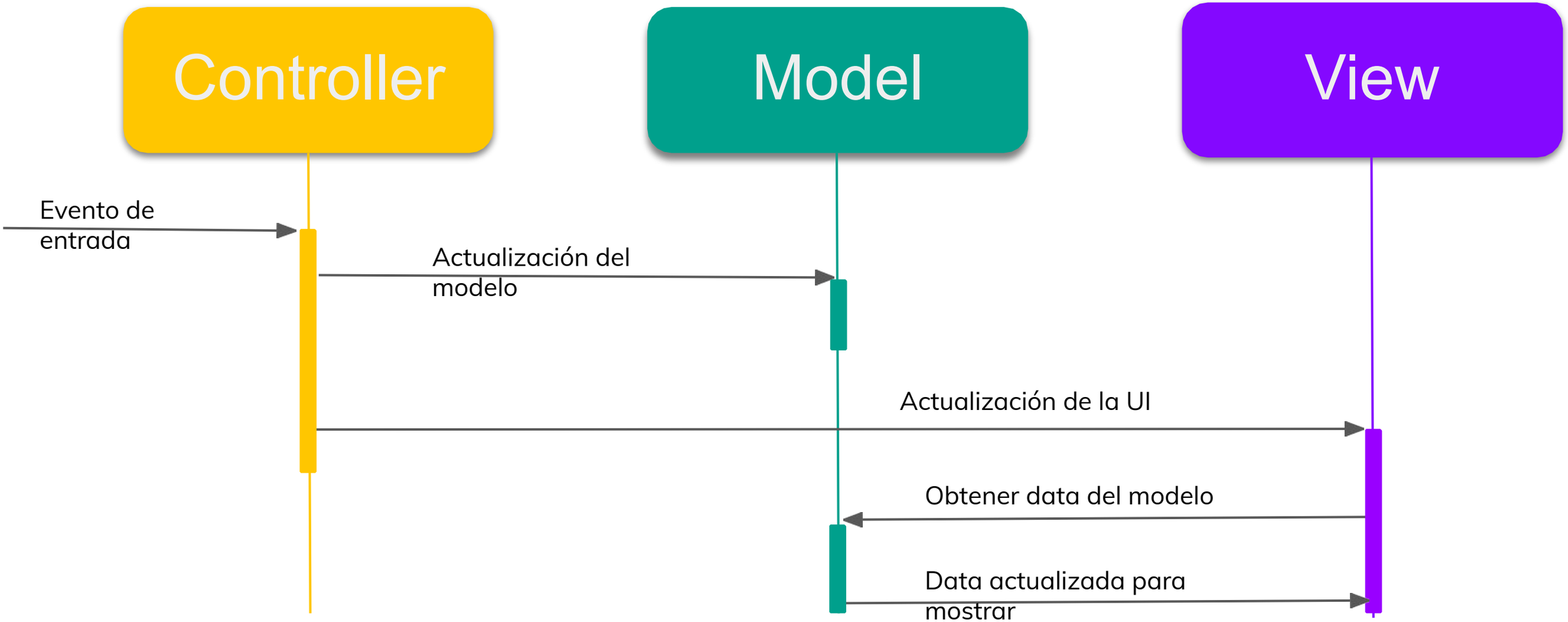
- Simplicidad: Cada componente define un único rol
- Testability: Código testeable para poder construir test unitarios
- Bajo costo de mantención: Agregar / quitar características (features) de forma fácil y limpia



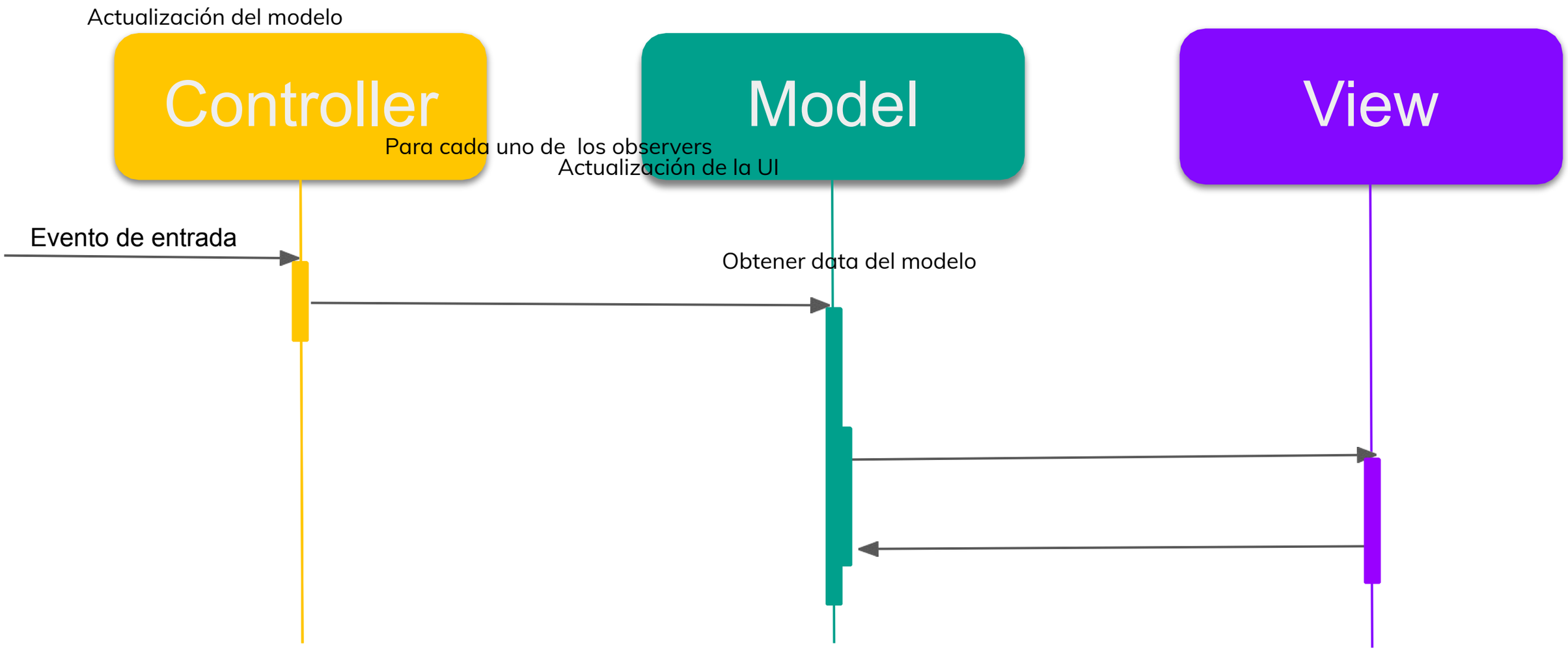
MVC en Android



MVC en Android



MVC en Android



Ejemplo Acumulador de donaciones

- Parte con 0 donaciones
- Al agregar una donación se debe:
  - Validar la nueva donación
  - Incrementar el total de donaciones
  - Entregar un mensaje de feedback al usuario

Ejemplo de Model

- Almacena la suma total de donaciones
- Guarda una donación sumándose al total

```
class DonationModel {
    var totalAmount = 0
        private set

    fun saveDonation(donationAmount:
Int): Boolean {
        totalAmount += donationAmount
        return true
    }
}
```

Ejemplo:
Activity (View + Controller)

- Actúa como controlador y vista
- Tiene lógica de validación
- Rápido de desarrollar
- Difícil de probar

```
public class MainActivity extends AppCompatActivity {

  private ActivityMainBinding binding;
  private DonationModel model;

  @Override
  protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    binding = ActivityMainBinding.inflate(getLayoutInflater());
    setContentView(binding.getRoot());
    model = new DonationModel();
    binding.btDonation.setOnClickListener(view -> saveDonation());


  public void saveDonation() {
    Integer amount = Integer.valueOf(binding.etNewDonation.getText().toString());
    if(amount <= 0) {
      displayMessage("El monto no es válido");
    } else {
      this.model.saveDonation(new Donation(amount));
      int totalAmount = this.model.getTotalAmount();

      binding.tvTotalAmount.setText(String.valueOf(totalAmount));
      displayMessage("Gracias por la donación!");

    binding.etNewDonation.setText("");
  }

  private void displayMessage(String message) {
    Toast.makeText(this, message, Toast.LENGTH_LONG).show();
  }
}
```

Lógica de la interfaz Responsable

```
String firstName = userModel.getFirstName();
String lastName = userModel.getLastName();

nameTextView.setText(lastName + ", " + firstName);
```

```
String displayName = userModel.getDisplayName();

nameTextView.setText(displayName);
```

View

Model

Ejemplo Controller

- Capa separada sin dependencia a la vista
- Capa separada del framework de Android
- Simple de probar

```
class Controller {

    // Comunico en el Model
    private val model : DonationModel
    init {
        model= DonationModel()
    }

    // llamo la funcion de model guardar
    fun saveDonation(amount: String): Boolean {

        return  model.saveDonation(amount)
    }


    // llamo la funcion de model muestra las
donaciones
    fun totalDonation(): Int {

        return model.getDonationAMount()
    }
}
```

Ejemplo: Activity(View)

- Capa separada sin dependencia a la vista
- Capa separada del framework de Android
- Simple de probar

```
class MainActivity : AppCompatActivity() {

   private lateinit var binding: ActivityMainBinding
   private lateinit var  controller: Controller

   override fun onCreate(savedInstanceState: Bundle?) {
      super.onCreate(savedInstanceState)
      binding = ActivityMainBinding.inflate(layoutInflater)
      setContentView(binding.root)
      // inicializamos controller

      controller = Controller()

      binding.button.setOnClickListener{ makeDonation() }
         }
   private  fun makeDonation(){
      val donation = controller.saveDonation(binding.tvDonation.text.toString())
      if(donation) {

         val totalDonation = controller.totalDonation()
         // recibo el total de las donaciones
         val total= getString(R.string.total_donaciones,totalDonation.toString())
         binding.tvTotal.text= total
         val toast = Toast.makeText(applicationContext, "Donación exitosa", Toast.LENGTH_LONG)
         toast.show()

      }
else{
         val toast1 = Toast.makeText(
            applicationContext, "Donación Fallida",
            Toast.LENGTH_LONG)
         toast1.show()

          }

      }
}
```

Realiza la siguiente Aplicación

1. **Modelo (Model)**:
   - Crear una clase DonacionModel que maneje el total de las donaciones.
   - Implementar un método agregarDonacion(double monto) para sumar nuevas donaciones.
   - Implementar un método obtenerTotal() para obtener el total acumulado.
2. **Vista (View)**:
   - Crear una interfaz de usuario con un EditText para ingresar el monto de la donación.
   - Agregar un Button para que el usuario registre la donación.
   - Usar un TextView para mostrar el total acumulado de las donaciones.
3. **Controlador (Controller)**:
   - En la actividad (MainActivity), inicializar el Modelo y la Vista.
   - Configurar un OnClickListener para el botón que capture el monto ingresado y lo pase al Modelo.
   - Actualizar la Vista con el nuevo total después de cada donación.

Este enfoque divide el código en tres componentes, asegurando que cada parte tenga una responsabilidad clara y facilitando el mantenimiento y la escalabilidad.

TIEMPO 45 minutos

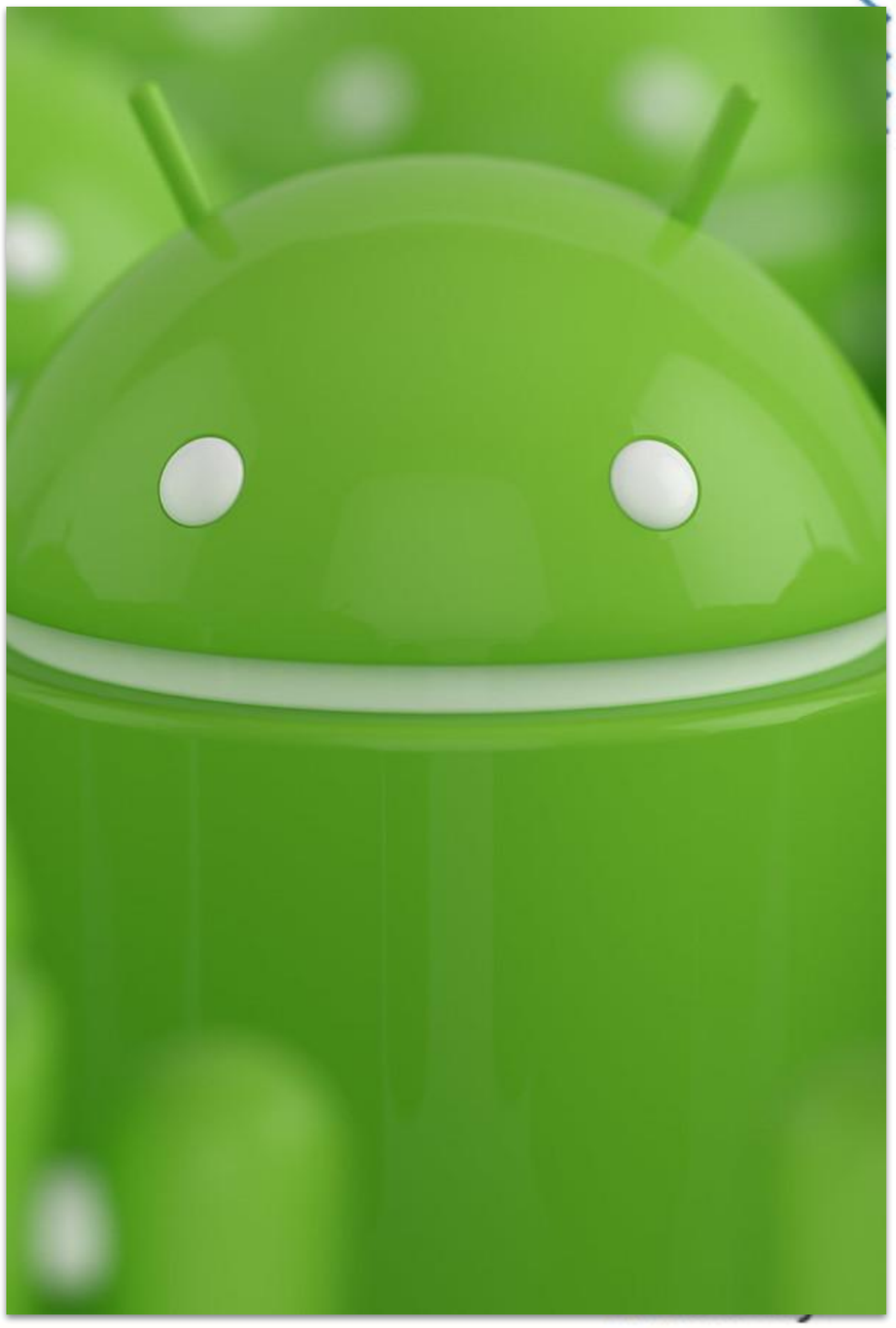
SOLUCIÓN

https://github.com/Mariocanedo/MVC_MVP_EXAMPLE_VERSIONFINAL

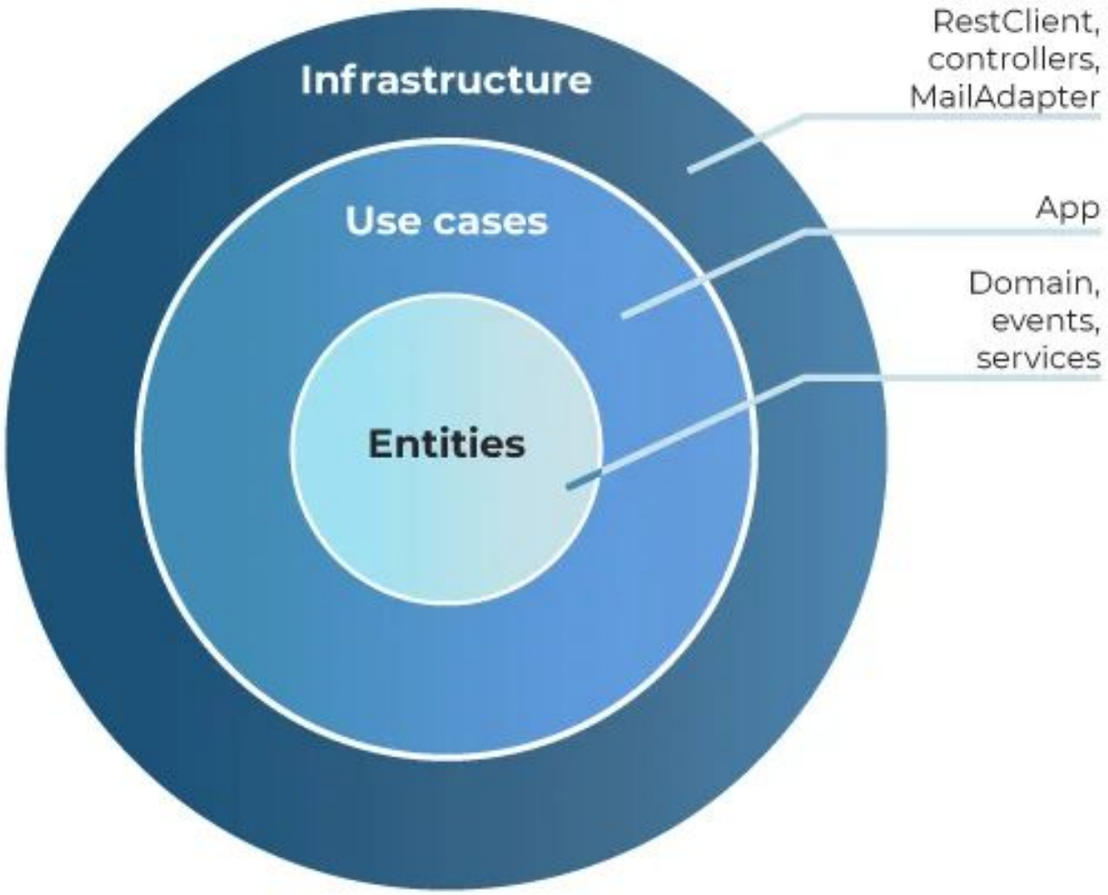
› Clean Arquitectura

Arquitectura Limpia (Clean Architecture)

Este patrón está diseñado para crear aplicaciones Android altamente mantenibles y testables. Divide la aplicación en capas bien definidas, incluyendo la Capa de Presentación (UI), Capa de Dominio (lógica empresarial) y Capa de Datos (acceso a datos). Cada capa tiene su propia responsabilidad y dependencias unidireccionales. Muchas veces se usa Clean Architecture junto con otro de los patrones mencionados anteriormente para ganar todas las ventajas de ambos patrones



Arquitectura Limpia (Clean Architecture)



¿Alguna consulta?

alkemy

Resumen

¿Qué logramos en esta clase?

- ✓ *Qué son los patrones de arquitectura*
- ✓ *Ventajas de usarlos*
- ✓ *Patrones de diseño*
- ✓ *MVC*

< ¡Muchas gracias! >

alkemy